

A PBL-I Synopsis on

Personal Finance Tracker

Submitted to Manipal University Jaipur

Towards the partial fulfilment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In Computers Science and Engineering

2025-2026

By

Piyush Prakash Sharma
2427030222



MANIPAL UNIVERSITY
JAIPUR

.

Under the Guidance of

Ms.Harshika Mathur

Sign of Supervisor

Department of Computer Science and Engineering
School of Computer Science and Engineering
Manipal University Jaipur
Jaipur, Rajasthan

I. Introduction to Problem (What, Why, How)

The "What"

When a person tries to manage their finances with modern digital payment apps, they are often met with platforms that actively encourage them to spend more rather than save. FinTracker is a personal finance and budgeting application designed to actively disincentivize overspending. It functions as a mix of a finance app and Duolingo—instead of rewarding users with cashbacks, it rewards them with psychological momentum (savings streaks). It uses feedback loops and the fear of losing that momentum to force accountability, helping users set goals and build genuine financial responsibility.

The "Why"

A significant gap exists in the current personal finance and payments market.

- **Commercial Payment Apps** (e.g., GPay, PhonePe): These platforms often scrape data and rely on toxic positive reinforcement, such as coupons and cashbacks, to encourage continuous spending.
- **Generic Budgeting Tools:** Traditional trackers rely on passive pie charts and lack the visceral emotional feedback necessary to create true financial awareness. They do not hold users accountable on a daily basis when they exceed their limits.

The "How"

The core idea of the application is :

Whenever you spend your money through online methods the app reads your transaction through messages and then notifies you how close you are to reaching your monthly/daily budget limit sort of like breaking the "Savings streak" "If by the end of the day you exceeded your daily budget limit then the app can reset your streak or give you an option to keep your streak by spending less the next day. It will also provide user with downloadable detailed reports of their transactions with charts and budget analysis

II. Literature Review

1. Premium Cloud SaaS (e.g., YNAB, Copilot Money)

- **Pros:** Excellent interfaces; uses Machine Learning for auto-categorization.
- **Cons:** High paywalls (avg. \$100/yr) and requires giving live bank credentials to the cloud, creating privacy risks.

2. Open-Source & Local Tools (e.g., GnuCash, Excel)

- **Pros:** 100% free and offers complete offline privacy.
- **Cons:** Extremely high friction. Requires tedious manual data entry with zero automation.

3. Modern UPI Apps (e.g., GPay, PhonePe)

- **Pros:** Frictionless and highly convenient.
- **Cons:** Employs "toxic positive reinforcement" (scratch cards, cashbacks) to actively encourage continuous spending.

III. Comparative Study

Application Type / Competitor	Cost	Privacy / Data Security	Categorization Method	Behavioral Focus
Cloud SaaS (<i>YNAB, Monarch</i>)	✗ High Subscription	✗ Cloud (Requires Bank Sync)	⚠ Rule-based / Manual	⚠ Passive Tracking
Copilot Money	✗ High Subscription	✗ Cloud (Requires Bank Sync)	☑ AI / Machine Learning	⚠ Passive Tracking
UPI Apps (<i>GPay, PhonePe</i>)	☑ Free	✗ Cloud (Monetizes Data)	⚠ Basic / Vendor level	✗ Toxic Positivity (Encourages Spending)
Open Source (<i>GnuCash</i>)	☑ Free	☑ 100% Local / Offline	✗ Strictly Manual Entry	✗ None
FinTracker (Proposed Solution)	☑ Free	☑ 100% Local (CSV Upload)	☑ ML-Powered (Naive Bayes)	☑ Aggressive Accountability (Streaks)

IV. Problem Statement

We are living in the era of frictionless spending. With the dawn of UPI apps reducing transactions to a mere tap, the average Indian now spends nearly **[15-20%]** more than they did in the cash era an increase that holds true even when you account for inflation and rising purchasing power. We've **optimized for convenience**, but we've completely **erased financial accountability**.

V. Objective

This application will be designed to actively **disincentivize overspending**. Unlike commercial products (Gpay ,PhonePe etc) that scrape data and use toxic positive reinforcement like cashbacks and coupons etc. to encourage spending, FinTracker focuses solely on helping you set goals and build financial responsibility. It is sort of a mix of a finance app and duolingo .The same way Duolingo doesn't reward you with money; it rewards you with psychological momentum (the Streak), and more importantly, it uses the fear of losing that momentum (passive-aggressive notifications) to force you to behave, it also relies on feedback loops and daily/monthly saving streaks to hold you accountable.

VI. Planning of Work / Proposed Solution

PBL 1 Implementation :

The current system is a functional Python script executing a linear data pipeline:

1. **Input :** The script accepts a raw CSV bank statement.
2. **Cleaning:** The `clean_amount_column` function sanitizes currency strings (removing '₹', commas) and standardizes data types.
3. **Categorization:** A keyword-matching algorithm maps transaction descriptions to categories (e.g., "Swiggy" → "Food") using a predefined dictionary.
4. **Reporting:** The `openpyxl` library generates a structured .xlsx file containing a transaction log, a category summary table, and an embedded Pie Chart.

PBL 2 Implementation

Step 1: Local Data Ingestion (Input)

- User defines a monthly budget limit via the UI.
- User uploads raw, unstructured bank transaction data (CSV format).
- Key Tech: Flask API (/api/upload) handles the local, offline transfer.

Step 2: The Machine Learning Pipeline (Processing)

- Data is cleaned (stripping symbols like ₹, handling negative values).
- Text descriptions are vectorized using TF-IDF.
- The Naive Bayes NLP Classifier predicts the exact expense category (e.g., Food, Utilities) without requiring manual rules.

Step 3: Behavioral Analysis (Logic)

- The system calculates the total spend vs. the user's predefined budget threshold.
- The emotional feedback engine determines the state of the user's "Savings Streak" and generates the appropriate pushback notification.

Step 4: Multi-Channel Output (Visualization)

- Frontend: Real-time JSON data updates the isometric React dashboard (animating SVG rings and panels).
- Backend: openpyxl generates a formatted, downloadable Excel report complete with color-coded summaries and embedded charts.

LIBRARIES USED :

1. Machine Learning & AI

- **scikit-learn:** The core brain of the project. It is used to build the TF-IDF Vectorizer (which turns transaction text into numbers) and the Multinomial Naive Bayes Classifier (which predicts the expense category).
- **joblib:** Used for model serialization. After `train_model.py` finishes training the AI, joblib saves the trained models into the `.pkl` files (`expense_model.pkl` and `expense_vectorizer.pkl`) so the Flask app can load them instantly without retraining.

2. Data Processing & Output

- **pandas:** The industry standard for data manipulation. It handles reading the raw, unstructured CSV uploads, cleaning the data (like removing currency symbols), and preparing the dataframes for the ML model.
- **openpyxl:** The engine behind your Excel generation. It takes the categorized data and writes it into a fully formatted `.xlsx` file, complete with color-coding and embedded pie charts for the final user report.

3. Backend Web Framework

- **Flask:** A lightweight Python web framework. It acts as the bridge between your machine learning Python code and your frontend user interface. It serves the dashboard and handles the `/api/upload` route that receives the user's CSV.

4. Synthetic Data Generation (Testing)

- **Faker:** Used specifically in `generate_large_csv.py`. It generates highly realistic, randomized synthetic data (fake names, transaction amounts, and descriptions) to create the `test_transactions.csv` file. This proves your app can handle large datasets (500+ rows) without needing real, sensitive bank data.

Future Enhancements

1. WhatsApp Bot Integration for Frictionless Manual Entry

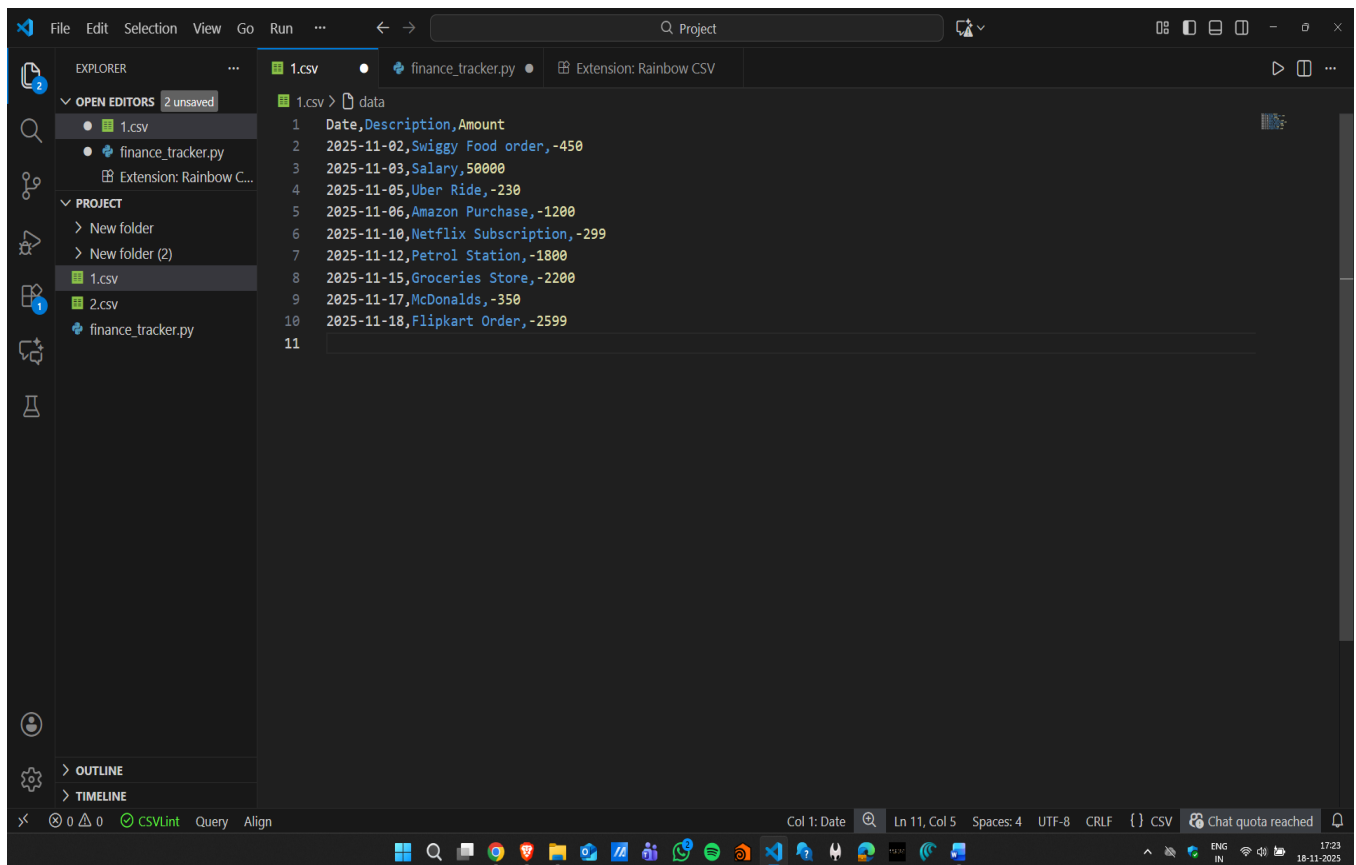
- The Concept: Deploying a conversational AI bot via the WhatsApp Business API to handle offline, cash-based, or unrecorded transactions. Users can simply text "Spent ₹300 on auto" to the bot, which passes the data to the Naive Bayes model.
- The "Why": Behavioral friction is the enemy of financial tracking. Since users already spend hours daily on social messaging apps, moving the input mechanism to WhatsApp eliminates the hurdle of opening a separate app. It meets the user where they already are.

2. Real-Time Device Notification & SMS Parsing

- The Concept: Transitioning from static CSV uploads to a local, on-device listener that intercepts and parses bank transaction SMS and UPI push notifications in real time.
- The "Why": Immediate feedback drives behavioral change. By capturing transactions the second they occur, the app can instantly trigger "savings streak" alerts and pushback notifications, closing the psychological gap between the act of spending and the awareness of the budget limit—all without compromising privacy by connecting to cloud banking APIs.

VII. Screenshots

PBL 1:

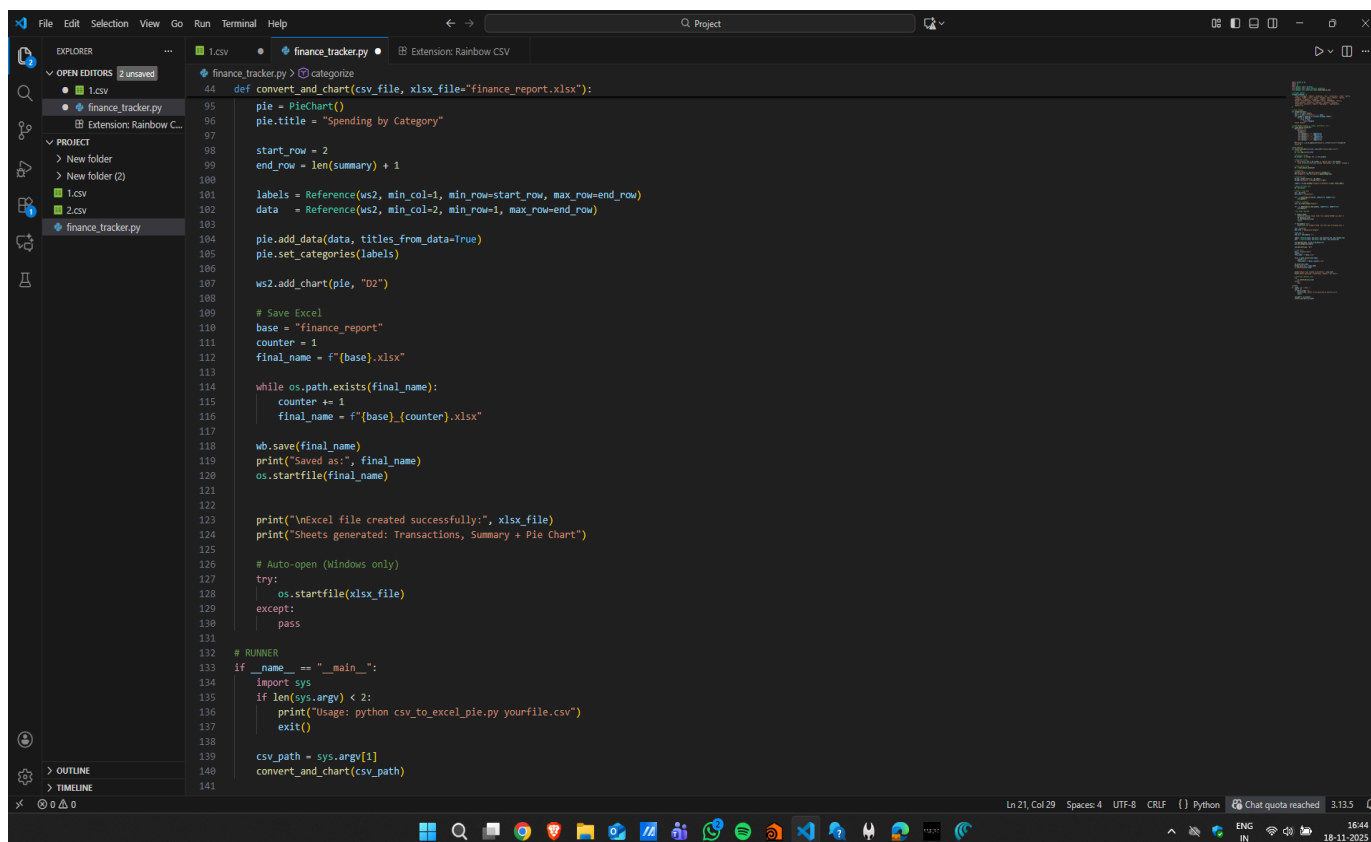
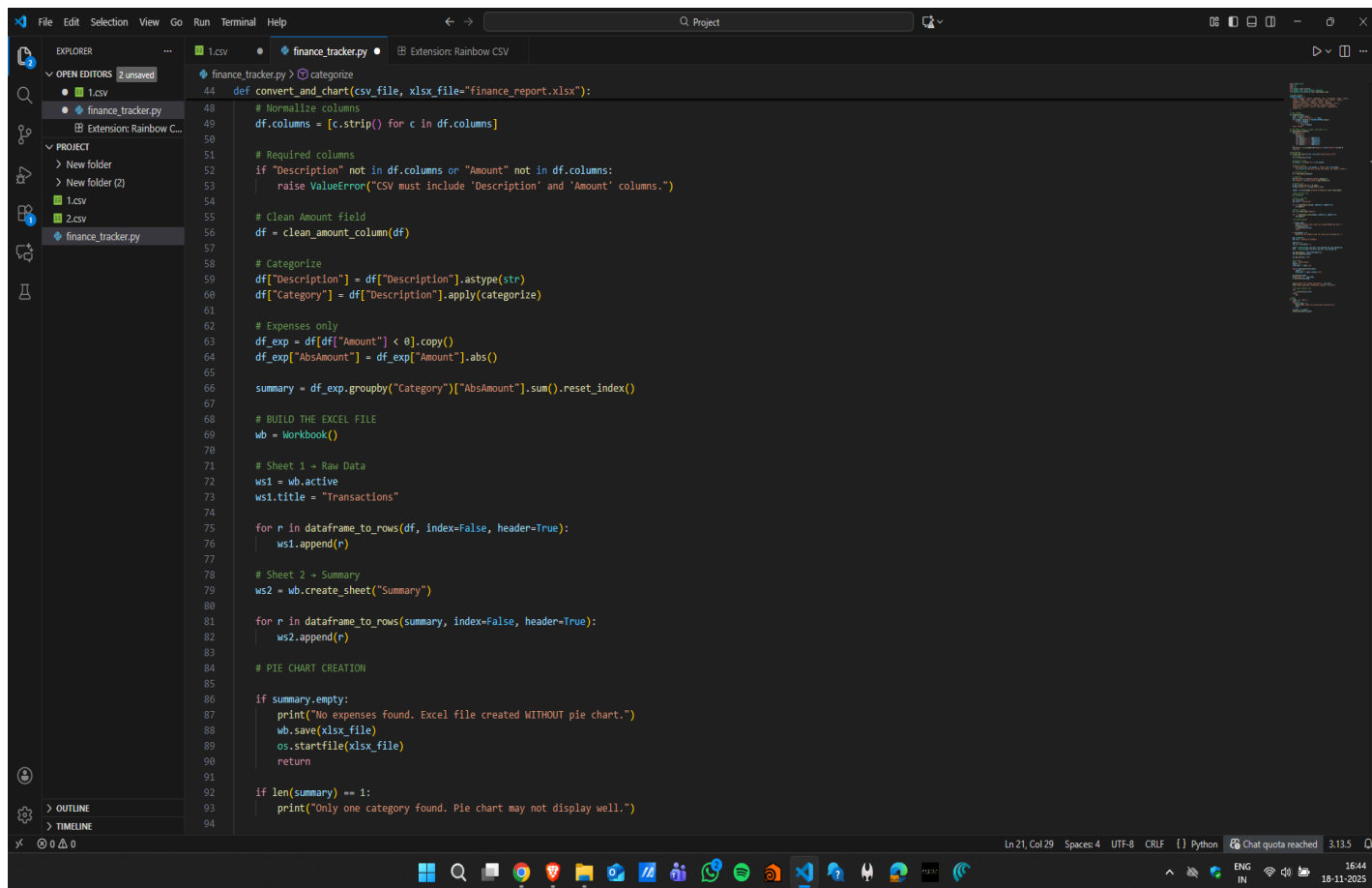


This screenshot shows a Visual Studio Code editor window with a CSV file named '2.csv' open. The file contains a list of transactions with columns for Date, Description, and Amount. The transactions include various expenses such as salary, groceries, rent, and utility bills. The interface includes the Explorer sidebar on the left, the Run and Debug sidebar on the right, and a status bar at the bottom.

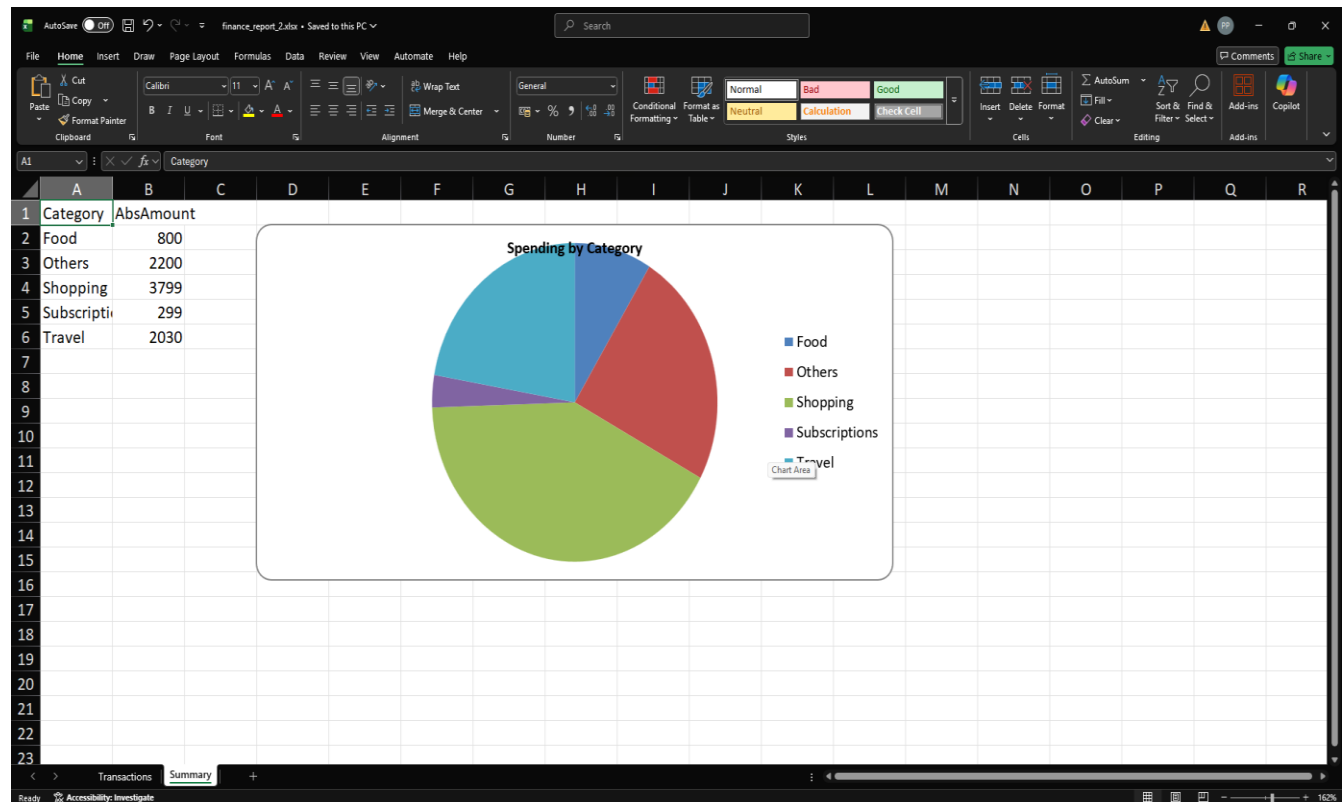
```
1 Date,Description,Amount
2 2025-11-01,Salary - Company XY7,90000.0
3 2025-11-02,KFC,-282.0
4 2025-11-02,Gym Membership,-2599.0
5 2025-11-03,Monthly Rent - Apartment,-25000.0
6 2025-11-03,Intercity Train Ticket,-531.0
7 2025-11-04,Big Bazaar Groceries,-1602.0
8 2025-11-04,Laundry,-154.0
9 2025-11-05,Home Loan EMI,-15000.0
10 2025-11-05,Petrol Station,-2569.0
11 2025-11-06,Swiggy,-345.0
12 2025-11-07,Electricity Bill,-3200.0
13 2025-11-08,Wmart Groceries,-2503.0
14 2025-11-08,Movie Tickets,-800.0
15 2025-11-09,Internet Bill,-999.0
16 2025-11-10,Zomato,-488.0
17 2025-11-11,Netflix Subscription,-299.0
18 2025-11-11,Spotify Subscription,-149.0
19 2025-11-12,Intercity Train Ticket,-649.0
20 2025-11-13,Cafe Coffee Day,-866.0
21 2025-11-13,Pharmacy,-450.0
22 2025-11-14,Local Supermarket,-2071.0
23 2025-11-14,Parking Fee,-439.0
24 2025-11-15,Flipkart Purchase - Clothes,-4999.0
25 2025-11-16,Petrol Station,-2318.0
26 2025-11-17,Uber Ride,-480.0
27 2025-11-17,Parking Fee,-417.0
28 2025-11-18,Swiggy,-824.0
29 2025-11-18,Refund - Merchant,1500.0
30 2025-11-19,Amazon Purchase - Electronics,-12499.0
31 2025-11-20,Big Bazaar Groceries,-4271.0
32 2025-11-20,Laundry,-668.0
33 2025-11-21,Myntra Purchase,-3599.0
34 2025-11-22,Zomato,-983.0
35 2025-11-23,Office Lunch Reimbursement (received),1200.0
36 2025-11-24,Intercity Train Ticket,-776.0
37 2025-11-25,Petrol Station,-2400.0
38 2025-11-26,Insurance Premium,-1999.0
39 2025-11-26,Domestic Flight Booking,-8999.0
40 2025-11-27,Big Bazaar Groceries,-3918.0
41 2025-11-28,Frind Transfer (received),3000.0
42 2025-11-29,Cafe - Workday,-968.0
43 2025-11-30,Laundry,-94.0
44
```

This screenshot shows a Visual Studio Code editor window with a Python script named 'finance_tracker.py' open. The script is designed to process a CSV file, categorize transactions, and generate a report. It includes functions for cleaning the data, categorizing it based on keywords, and converting it into a chart. The script uses libraries like pandas, re, and openpyxl.

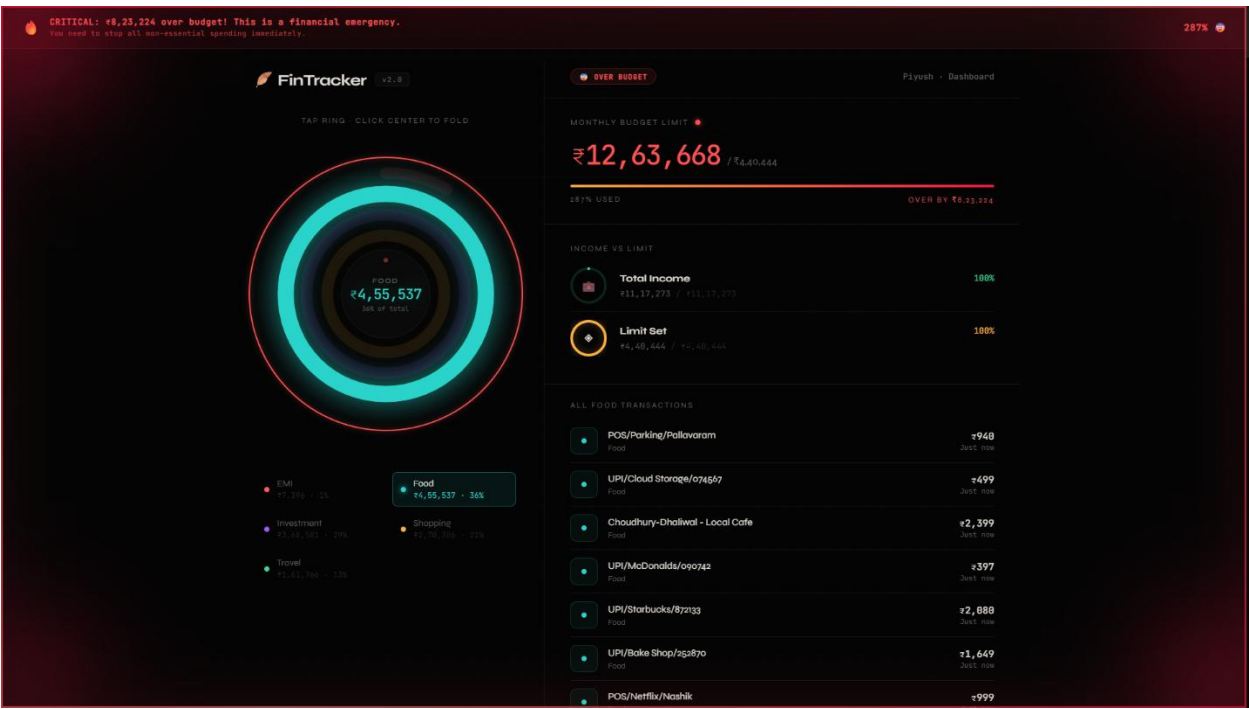
```
1 import pandas as pd
2 import re
3 import os
4 from openpyxl import Workbook
5 from openpyxl.chart import PieChart, Reference
6 from openpyxl.utils.dataframe import dataframe_to_rows
7
8 # CATEGORY MAPPING
9 CATEGORY_KEYWORDS = {
10     "Food": ["swiggy", "zomato", "mcdonald", "kfc", "restaurant", "food", "cafe"],
11     "Travel": ["uber", "ola", "train", "flight", "bus", "petrol", "auto"],
12     "Shopping": ["amazon", "flipkart", "myntra", "ajio"],
13     "Bills": ["electricity", "internet", "wifi", "broadband", "bill"],
14     "Subscriptions": ["netflix", "spotify", "hotstar", "prime"],
15     "Groceries": ["grocery", "dmart", "big bazaar", "supermarket"],
16     "Others": []
17 }
18
19 # CLEAN CATEGORY
20 def categorize(text):
21     text = str(text).lower()
22     text = re.sub(r"[a-z0-9s]", " ", text)
23     for category, keywords in CATEGORY_KEYWORDS.items():
24         for kw in keywords:
25             if kw in text:
26                 return category
27     return "Others"
28
29 # CLEAN AMOUNT (handles ₹, commas, parentheses, etc.)
30 def clean_amount_column(df):
31     df["Amount"] = (
32         df["Amount"]
33         .astype(str)
34         .str.replace("₹", "", regex=False)
35         .str.replace("(", "", regex=False)
36         .str.replace(")", "", regex=False)
37         .str.replace(",", "", regex=False)
38         .str.replace(" ", "", regex=False)
39     )
40     df["Amount"] = pd.to_numeric(df["Amount"], errors="coerce").fillna(0.0)
41     return df
42
43 # MAIN FUNCTION
44 def convert_and_chart(csv_file, xlsx_file="finance_report.xlsx"):
45     # Load CSV
46     df = pd.read_csv(csv_file)
47
48     # Normalize columns
```



Date		Description	Amount	Category
2025-11-02		Swiggy Food order	-450	Food
2025-11-03		Salary	50000	Others
2025-11-05		Uber Ride	-230	Travel
2025-11-06		Amazon Purchase	-1200	Shopping
2025-11-10		Netflix Subscription	-299	Subscriptions
2025-11-12		Petrol Station	-1800	Travel
2025-11-15		Groceries Store	-2200	Others
2025-11-17		McDonalds	-350	Food
2025-11-18		Flipkart Order	-2599	Shopping



The image is a composite of two screenshots of the FinTracker mobile application. The top screenshot shows the app's splash screen, which is dark-themed. It features the FinTracker logo (a stylized orange leaf) and the text "FinTracker" in white. Below the logo, it asks "What was your monthly budget limit?". There is a text input field containing "₹ e.g. 50000" and a button labeled "INITIALIZE DASHBOARD -". The bottom screenshot shows the app's main dashboard for a user named "Piyesh". The dashboard has a dark background with orange accents. At the top, there is a "CAUTION" banner. The main section displays the "MONTHLY BUDGET LIMIT" as ₹13,54,655 / ₹16,00,000, with a progress bar indicating 97% used and ₹45,344 remaining. Below this, the "INCOME VS LIMIT" section shows "Total Income" at ₹16,39,673 (100%) and "Limit Set" at ₹14,00,000 (100%). The "RECENT ACTIVITY" section lists several transactions: POS/Dominos/Sri Ganganganagar (₹27), Transfer from Binola Bhargava (₹20,886), UPI/Fuel/Kozhik (₹2,257), Gala-Chillon - Car Rental (₹4,568), POS/Donation/Solapur (₹2,175), Palan Group - YouTube Premium (₹299), and POS/YouTube Premium/Kozhikode (₹499). On the left side of the dashboard, there is a large circular progress indicator showing 97% spent, with the text "TOTAL SPENT ₹13,54,656" and "97% spent" inside the circle.



VIII. Bibliography

1. **Pandas Development Team.** (2024). *pandas: powerful Python data analysis toolkit*. Available: <https://pandas.pydata.org>
2. **Gazoni, E. & Clark, C.** (2024). *openpyxl: A Python library to read/write Excel 2010 xlsx/xlsm files*. Available: <https://openpyxl.readthedocs.io>
3. **Pedregosa, F. et al.** (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
4. **Plaid Inc.** (2024). *Plaid API Documentation*. Available: <https://plaid.com/docs/>
5. **You Need A Budget (YNAB).** (2024). *The Four Rules of Budgeting*. Available: <https://www.ynab.com/the-four-rules>